

RAG Pipeline

Implementation Lab Guide

LangChain · FAISS · OpenAI · Streamlit · Multi-Model Comparison

LAB 1	LAB 2	LAB 3	LAB 4
Full RAG Pipeline LangChain + FAISS + OpenAI	Irrelevant Retrieval RetrievalQA Observation	Document QA UI Streamlit + LangChain	Multi-Model Compare OpenAI · Claude · Gemini

Environment	AWS EC2 Ubuntu · Code-Server IDE
Prerequisites	Python 3.10+ · OpenAI API Key · pip
Duration	Approx. 4–5 hours (all labs combined)
Difficulty	Intermediate → Advanced
Version	LangChain 0.3.x · FAISS-cpu · Streamlit 1.x

Table of Contents

Complete Lab Navigation

01	Environment Setup & Prerequisites	3
02	LAB 1 — Full RAG Pipeline with LangChain + FAISS	5
2.1	Theory: RAG Architecture Deep Dive	5
2.2	Project Structure & Dependencies	6
2.3	Document Ingestion & Text Splitting	7
2.4	FAISS Vector Store Creation	8
2.5	RAG Chain Implementation	9
2.6	Running Queries & Testing	10
03	LAB 2 — Irrelevant Retrieval Observation	11
3.1	Theory: Why Retrieval Goes Wrong	11
3.2	Setting Up Policy Documents	12
3.3	Inducing & Observing Bad Retrieval	13
3.4	Fixes: Metadata Filtering & Re-ranking	14
04	LAB 3 — Streamlit Document QA UI	16
4.1	Streamlit Architecture Overview	16
4.2	File Upload & Processing	17
4.3	Session State & Conversational Memory	18
4.4	Complete App Code & Deployment	19
05	LAB 4 — Multi-Model Answer Comparison	22
5.1	Theory: Why Compare Models?	22
5.2	OpenAI GPT-4 Integration	23
5.3	Anthropic Claude Integration	24
5.4	Google Gemini Integration	25
5.5	Side-by-Side Comparison App	26
06	Troubleshooting Guide	29
07	Capstone Challenge	30

MODULE 00

Environment Setup & Prerequisites

Prepare your AWS EC2 Code-Server workspace

1.1 Infrastructure Overview

Each participant has been provisioned with an **AWS EC2 Ubuntu 22.04 instance** with **Code-Server** (VS Code in browser) pre-installed. All labs are executed inside this environment. You will access it via your browser at **http://<EC2-IP>:8080**.

Component	Version / Detail	Purpose
AWS EC2	t3.medium / Ubuntu 22.04	Lab execution environment
Code-Server	v4.x (VS Code in browser)	IDE & terminal access
Python	3.10 or 3.11	Runtime for all labs
pip	Latest (23.x+)	Package installer
OpenAI API Key	Provided / participant-owned	LLM & embedding calls
Anthropic API Key	Required for Lab 4	Claude model access
Google AI API Key	Required for Lab 4	Gemini model access

1.2 Step-by-Step Environment Setup

Step 1 — Open a terminal in Code-Server

Click the hamburger menu → Terminal → New Terminal. All commands below run in this terminal.

Step 2 — Create an isolated Python virtual environment

BASH

```
# Create project directory
mkdir -p ~/rag-labs && cd ~/rag-labs

# Create virtual environment
python3 -m venv venv

# Activate (do this every time you open a new terminal)
source venv/bin/activate

# Confirm python path
which python # should show ~/rag-labs/venv/bin/python
```

Step 3 — Install all required packages

BASH

```
# Core RAG stack
pip install langchain langchain-community langchain-openai
pip install faiss-cpu openai tiktoken

# Document loaders
pip install pypdf docx2txt unstructured

# Streamlit for Lab 3
pip install streamlit

# Multi-model APIs for Lab 4
pip install anthropic google-generativeai

# Utilities
pip install python-dotenv pandas
```

Step 4 — Configure API keys via .env file

■ NOTE

Never hard-code API keys in source files. Always use environment variables or .env files. The .env file must be in .gitignore if you use version control.

BASH

```
# Create .env file in project root
cat > ~/rag-labs/.env << 'EOF'
OPENAI_API_KEY=sk-your-openai-key-here
ANTHROPIC_API_KEY=sk-ant-your-anthropic-key-here
GOOGLE_API_KEY=Aiza-your-google-key-here
EOF

# Verify it was created
cat ~/rag-labs/.env
```

Step 5 — Verify installation

BASH

```
python -c "
import langchain, faiss, openai, streamlit
print('LangChain:', langchain.__version__)
print('FAISS: OK')
print('OpenAI:', openai.__version__)
print('Streamlit:', streamlit.__version__)
print('All packages installed successfully!')
"
```

✓ TIP

Expected output:
LangChain: 0.3.x
FAISS: OK
OpenAI: 1.x.x
Streamlit: 1.x.x
All packages installed successfully!

MODULE 01

LAB 1 — Full RAG Pipeline with LangChain + FAISS

Build an end-to-end Retrieval-Augmented Generation system from scratch

■ Lab Objectives

- ✓ Understand the three-stage RAG pipeline: Ingestion → Retrieval → Generation
- ✓ Load and split PDF/text documents using LangChain document loaders
- ✓ Generate embeddings using OpenAI text-embedding-3-small
- ✓ Build and persist a FAISS vector index for similarity search
- ✓ Construct a RetrievalQA chain to answer questions from retrieved context
- ✓ Evaluate answer quality and trace retrieved source chunks

2.1 Theory: RAG Architecture

Retrieval-Augmented Generation (RAG) is a technique that grounds LLM responses in external knowledge by retrieving relevant document chunks at query time. This overcomes the LLM's knowledge cutoff and hallucination tendencies.

Stage	Components	What Happens
1 — Ingestion	Document Loader, Text Splitter, Embedding Model, Vector Store	Documents are loaded, chunked into segments, converted to embedding vectors, and stored in FAISS index
2 — Retrieval	FAISS Retriever, Similarity Search	User query is embedded; top-k most similar chunks are fetched from the vector store
3 — Generation	Prompt Template, LLM, Output Parser	Retrieved chunks + user query are combined in a prompt; LLM generates a grounded answer

2.2 Project Structure

FILE STRUCTURE

```
rag-labs/
├── lab1_rag_pipeline/
│   ├── documents/           # Place your PDF/TXT files here
│   │   ├── policy.pdf
│   │   └── handbook.txt
│   ├── faiss_index/         # Auto-created: persisted vector store
│   ├── rag_pipeline.py      # Main pipeline script
│   └── config.py            # Configuration & constants
├── .env
└── venv/
```

BASH

```
# Create directories
mkdir -p ~/rag-labs/lab1_rag_pipeline/documents
cd ~/rag-labs/lab1_rag_pipeline
```

2.3 Sample Documents — Create Policy Files

We'll create synthetic insurance/HR policy documents to simulate real-world enterprise RAG scenarios.

BASH

```
# Create documents/policy.txt
cat > documents/policy.txt << 'POLICY'
HEALTH INSURANCE POLICY – Section 3: Claims
All claims must be submitted within 90 days of service date.
Pre-authorization is required for surgeries above $5,000.
Mental health visits are covered up to 20 sessions per year.
Emergency room visits require notification within 48 hours.

LEAVE POLICY – Section 7: Sick Leave
Employees are entitled to 12 days sick leave per calendar year.
Medical certificate required for absences exceeding 3 consecutive days.
Unused sick leave cannot be carried forward to the next year.
POLICY
```

BASH

```
# Create documents/handbook.txt
cat > documents/handbook.txt << 'HANDBOOK'
EMPLOYEE HANDBOOK – IT Security Policy
Passwords must be changed every 90 days and contain 12+ characters.
VPN is mandatory when accessing company resources remotely.
Reporting a security incident: email security@company.com within 1 hour.
Personal devices require MDM enrollment before accessing company email.

EXPENSE REIMBURSEMENT POLICY
Travel expenses above $500 require prior manager approval.
Meal allowance is $75 per day during business travel.
Receipts must be submitted within 30 days via the expense portal.
HANDBOOK
```

2.4 config.py — Configuration Constants

PYTHON — config.py

```
# config.py
import os
from dotenv import load_dotenv

load_dotenv() # Loads .env from project root

# API Configuration
OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')

# Model settings
EMBEDDING_MODEL = 'text-embedding-3-small' # Cost-efficient embeddings
LLM_MODEL       = 'gpt-4o-mini'           # Fast, affordable LLM
LLM_TEMPERATURE = 0.0                     # Deterministic for QA

# RAG hyperparameters
CHUNK_SIZE      = 500 # Characters per chunk
CHUNK_OVERLAP   = 50  # Overlap between adjacent chunks
TOP_K           = 4    # Number of chunks to retrieve

# Paths
DOCS_DIR        = 'documents'
INDEX_PATH      = 'faiss_index'
```

2.5 rag_pipeline.py — Main Implementation

Part A: Imports and Initialization

PYTHON

```
# rag_pipeline.py
import os
from config import *

from langchain_community.document_loaders import (
    DirectoryLoader, TextLoader, PyPDFLoader
)
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
```

Part B: Document Loading and Splitting

PYTHON

```
def load_and_split_documents(docs_dir: str):
    """Load all .txt and .pdf files, split into chunks."""

    # Load .txt files
    txt_loader = DirectoryLoader(
        docs_dir,
        glob='**/*.txt',
        loader_cls=TextLoader,
        loader_kwargs={'encoding': 'utf-8'}
    )
    txt_docs = txt_loader.load()

    # Load .pdf files (if any)
    pdf_loader = DirectoryLoader(
        docs_dir,
        glob='**/*.pdf',
        loader_cls=PyPDFLoader
    )
    pdf_docs = pdf_loader.load()

    all_docs = txt_docs + pdf_docs
    print(f'Loaded {len(all_docs)} document(s)')

    # Split into chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=['\n\n', '\n', '.', ' ', ' ', '']
    )
    chunks = splitter.split_documents(all_docs)
    print(f'Created {len(chunks)} chunks')
    return chunks
```

Part C: FAISS Vector Store

PYTHON

```
def build_or_load_vectorstore(chunks=None):
    """Build FAISS index from chunks or load existing index."""

    embeddings = OpenAIEmbeddings(
        model=EMBEDDING_MODEL,
        openai_api_key=OPENAI_API_KEY
    )

    if os.path.exists(INDEX_PATH) and chunks is None:
        print('Loading existing FAISS index...')
        vectorstore = FAISS.load_local(
            INDEX_PATH,
            embeddings,
            allow_dangerous_deserialization=True
        )
        print('Index loaded!')
    else:
        print('Building new FAISS index...')
        vectorstore = FAISS.from_documents(chunks, embeddings)
        vectorstore.save_local(INDEX_PATH)
        print(f'Index saved to {INDEX_PATH}/')

    return vectorstore
```

Part D: RAG Chain Construction

PYTHON

```
def build_rag_chain(vectorstore):  
    """Construct a RetrievalQA chain with custom prompt."""  
  
    # Custom prompt enforces grounded answers  
    prompt_template = """You are a helpful assistant answering questions  
based ONLY on the provided context. If the answer is not in the context,  
say 'I don't have enough information to answer this question.'  
  
Context:  
{context}  
  
Question: {question}  
  
Answer: """  
  
    PROMPT = PromptTemplate(  
        template=prompt_template,  
        input_variables=['context', 'question']  
    )  
  
    llm = ChatOpenAI(  
        model=LLM_MODEL,  
        temperature=LLM_TEMPERATURE,  
        openai_api_key=OPENAI_API_KEY  
    )  
  
    retriever = vectorstore.as_retriever(  
        search_type='similarity',  
        search_kwargs={'k': TOP_K}  
    )  
  
    chain = RetrievalQA.from_chain_type(  
        llm=llm,  
        chain_type='stuff',          # Stuff all chunks into context  
        retriever=retriever,  
        return_source_documents=True, # Show which chunks were used  
        chain_type_kwargs={'prompt': PROMPT}  
    )  
    return chain
```

Part E: Main Function — Run Queries

PYTHON

```
def main():
    print('=== RAG Pipeline Demo ===')

    # Stage 1: Ingest
    chunks = load_and_split_documents(DOCS_DIR)

    # Stage 2: Build vector store
    vectorstore = build_or_load_vectorstore(chunks)

    # Stage 3: Build chain
    chain = build_rag_chain(vectorstore)

    # Test queries
    queries = [
        'How many sick leave days are employees entitled to?',
        'What is required before a major surgery?',
        'What is the password policy?',
        'How much is the meal allowance during travel?',
    ]

    for query in queries:
        print(f'\nQ: {query}')
        result = chain.invoke({'query': query})
        print(f'A: {result["result"]}')
        print('Sources:')
        for doc in result['source_documents']:
            src = doc.metadata.get('source', 'unknown')
            print(f' - {src}: {doc.page_content[:80]}...')

if __name__ == '__main__':
    main()
```

Run the pipeline

BASH

```
cd ~/rag-labs/lab1_rag_pipeline
python rag_pipeline.py
```

✓ TIP

Expected Output:

Loaded 2 document(s)

Created 8 chunks

Building new FAISS index...

Index saved to faiss_index/

Q: How many sick leave days are employees entitled to?

A: Employees are entitled to 12 days sick leave per calendar year.

Sources:

- documents/policy.txt: LEAVE POLICY — Section 7: Sick Leave...

MODULE 02

LAB 2 — Irrelevant Retrieval Observation

Ask questions on policy docs and observe how retrieval fails — then fix it

3.1 Theory: Why Retrieval Fails

A naive RAG pipeline retrieves chunks purely by **cosine similarity** between query and chunk embeddings. This can fail when: (a) out-of-domain queries are asked, (b) vocabulary mismatch occurs, or (c) the top-k chunks are all from the same document section, missing relevant context from other parts.

Failure Type	Root Cause	Symptom
Out-of-domain query	Query has no related chunks in the store	LLM hallucinates or says it cannot answer but retrieves random chunks
Vocabulary mismatch	Query uses synonyms not present in documents	Semantically similar but contextually wrong chunks are returned
Chunk boundary issue	Answer spans two adjacent chunks	Retrieved chunk has partial context; answer is incomplete
Top-k saturation	Multiple chunks from same section fill top-k	Diverse coverage lost; answer biased to one document section

3.2 Setting Up the Observation Script

BASH

```
# Create lab2 directory
mkdir -p ~/rag-labs/lab2_retrieval_observation
cd ~/rag-labs/lab2_retrieval_observation

# Reuse documents from lab1
cp -r ../lab1_rag_pipeline/documents .
cp ../lab1_rag_pipeline/config.py .
```

3.3 observe_retrieval.py — Full Observation Script

PYTHON — Part 1

```
# observe_retrieval.py
from config import *
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

def build_qa_chain():
    loader = DirectoryLoader('documents', glob='**/*.txt',
                           loader_cls=TextLoader,
                           loader_kwargs={'encoding': 'utf-8'})

    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=300, chunk_overlap=30 # Smaller chunks = more retrieval issues
    )
    chunks = splitter.split_documents(docs)

    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL,
                                   openai_api_key=OPENAI_API_KEY)
    vectorstore = FAISS.from_documents(chunks, embeddings)

    llm = ChatOpenAI(model=LLM_MODEL, temperature=0,
                     openai_api_key=OPENAI_API_KEY)

    prompt = PromptTemplate(
        template='Context:\n{context}\n\nQuestion: {question}\nAnswer:',
        input_variables=['context', 'question']
    )
    chain = RetrievalQA.from_chain_type(
        llm=llm, chain_type='stuff',
        retriever=vectorstore.as_retriever(search_kwargs={'k': 3}),
        return_source_documents=True,
        chain_type_kwargs={'prompt': prompt}
    )
    return chain, vectorstore
```

PYTHON — Part 2

```
def observe_query(chain, vectorstore, query, label):
    print(f'\n{'='*60}')
    print(f'TEST: {label}')
    print(f'Query: {query}')

    # Direct similarity search — shows raw retrieval
    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL,
                                    openai_api_key=OPENAI_API_KEY)
    raw_hits = vectorstore.similarity_search_with_score(query, k=3)
    print('\nRaw Retrieved Chunks (lower score = more similar):')
    for i, (doc, score) in enumerate(raw_hits):
        print(f' [{i+1}] Score: {score:.4f} | {doc.page_content[:100]}...')

    # Full chain answer
    result = chain.invoke({'query': query})
    print(f'\nLLM Answer: {result["result"]}')

if __name__ == '__main__':
    chain, vectorstore = build_qa_chain()

    # Test cases demonstrating retrieval issues
    test_cases = [
        ('How many sick leave days do employees get?',
         'RELEVANT — exact topic in document'),

        ('What is the capital of France?',
         'OUT-OF-DOMAIN — completely unrelated to policy docs'),

        ('How many annual leave days?',
         'VOCABULARY MISMATCH — doc says sick leave not annual leave'),

        ('Tell me about leave policies and expense policies',
         'MULTI-TOPIC — answer spans multiple document sections'),

        ('What happens if I miss the 90 day deadline?',
         'AMBIGUOUS — two different 90-day rules exist in docs'),
    ]

    for query, label in test_cases:
        observe_query(chain, vectorstore, query, label)
```

BASH

```
python observe_retrieval.py
```

■ NOTE

OBSERVATION: For 'What is the capital of France?' — the LLM will still retrieve chunks (usually about 'leave policies' since those have similar embedding distances). The answer will be fabricated or the LLM will say 'not in context', but notice WHICH chunks were retrieved — they are irrelevant. This is the core problem.

3.4 Fixes: Improving Retrieval Quality

Fix 1 — MMR (Maximal Marginal Relevance) for diversity

PYTHON — Fix 1

```
# Replace as_retriever() with MMR search to avoid top-k saturation
retriever = vectorstore.as_retriever(
    search_type='mmr',          # Diversified retrieval
    search_kwargs={
        'k': 4,                # Final chunks to return
        'fetch_k': 20,          # Candidates to consider
        'lambda_mult': 0.7      # 0=max diversity, 1=max relevance
    }
)
```

Fix 2 — Similarity score threshold to reject irrelevant queries

PYTHON — Fix 2

```
# Reject retrieval if similarity score is too low
retriever = vectorstore.as_retriever(
    search_type='similarity_score_threshold',
    search_kwargs={
        'k': 4,
        'score_threshold': 0.75 # Only return chunks with >= 75% similarity
    }
)

# In your chain, check if any documents were retrieved:
result = chain.invoke({'query': query})
if not result['source_documents']:
    print('No relevant documents found for this query.')
```

Fix 3 — Metadata filtering for domain-scoped retrieval

PYTHON — Fix 3

```
# Tag documents with metadata during loading
from langchain.schema import Document

# Add metadata when creating chunks
for chunk in chunks:
    if 'policy.txt' in chunk.metadata['source']:
        chunk.metadata['domain'] = 'hr_policy'
    elif 'handbook.txt' in chunk.metadata['source']:
        chunk.metadata['domain'] = 'it_security'

# Filter retrieval by domain
retriever = vectorstore.as_retriever(
    search_kwargs={
        'k': 4,
        'filter': {'domain': 'hr_policy'} # Only search HR docs
    }
)
```

MODULE 03

LAB 3 — Streamlit Document QA System

Build a production-ready web UI for document question answering

4.1 Architecture Overview

We'll build a multi-file document QA system with a clean Streamlit UI. Users upload PDF/TXT files, the system indexes them on the fly, and then supports conversational Q&A; with memory of previous turns.

Feature	Implementation
File Upload	st.file_uploader() — accepts PDF and TXT files
Real-time indexing	FAISS index built on upload, cached in session state
Conversational memory	ConversationBufferMemory preserves chat history
Source display	Expander shows retrieved document chunks with scores
Multiple files	DirectoryLoader indexes all uploaded files together
Clear history	Button resets session state and rebuilds index

4.2 Setup

BASH

```
mkdir -p ~/rag-labs/lab3_streamlit_qa/uploaded_docs
cd ~/rag-labs/lab3_streamlit_qa
cp ../lab1_rag_pipeline/config.py .
```

4.3 app.py — Complete Streamlit Application

Part A — Imports and Page Configuration

PYTHON — Part A

```
# app.py
import os, tempfile, shutil
import streamlit as st
from config import *

from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
from langchain.prompts import PromptTemplate

st.set_page_config(
    page_title='Document QA System',
    page_icon='■',
    layout='wide',
    initial_sidebar_state='expanded'
)
```

Part B — Custom CSS Styling

PYTHON — Part B

```
st.markdown('''
<style>
.main-header {
    background: linear-gradient(135deg, #0F172A, #1E293B);
    padding: 1.5rem; border-radius: 8px;
    border-left: 4px solid #6366F1; margin-bottom: 1rem;
}
.chat-user { background: #EEF2FF; padding: 0.8rem;
              border-radius: 8px; margin: 0.5rem 0; }
.chat-bot  { background: #F1F5F9; padding: 0.8rem;
              border-radius: 8px; margin: 0.5rem 0; }
.source-box { background: #ECFDF5; padding: 0.5rem;
              border-radius: 4px; font-size: 0.8rem; }
</style>
''', unsafe_allow_html=True)
```

Part C — Document Processing Functions

PYTHON — Part C

```
@st.cache_resource(show_spinner='Indexing documents...')
def process_uploaded_files(file_paths_tuple):
    """Load, split, embed, and index uploaded files."""
    all_chunks = []
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP
    )

    for fpath in file_paths_tuple:
        ext = os.path.splitext(fpath)[1].lower()
        try:
            if ext == '.pdf':
                loader = PyPDFLoader(fpath)
            else:
                loader = TextLoader(fpath, encoding='utf-8')
            docs = loader.load()
            chunks = splitter.split_documents(docs)
            all_chunks.extend(chunks)
        except Exception as e:
            st.error(f'Error loading {fpath}: {e}')

    if not all_chunks:
        return None

    embeddings = OpenAIEmbeddings(
        model=EMBEDDING_MODEL, openai_api_key=OPENAI_API_KEY
    )
    vectorstore = FAISS.from_documents(all_chunks, embeddings)
    return vectorstore
```

Part D — Build Conversational Chain


```
# Main area
st.markdown('<div class="main-header">', unsafe_allow_html=True)
st.title('Document QA System')
st.markdown('Ask questions about your uploaded policy documents.')
st.markdown('</div>', unsafe_allow_html=True)

# Initialize session state
if 'messages' not in st.session_state:
    st.session_state.messages = []
if 'chain' not in st.session_state:
    st.session_state.chain = None

# Process uploaded files
if uploaded:
    tmp_dir = tempfile.mkdtemp()
    file_paths = []
    for uf in uploaded:
        fpath = os.path.join(tmp_dir, uf.name)
        with open(fpath, 'wb') as f:
            f.write(uf.getvalue())
        file_paths.append(fpath)

    file_key = tuple(sorted([f.name for f in uploaded]))

    if ('indexed_files' not in st.session_state or
        st.session_state.indexed_files != file_key):
        vectorstore = process_uploaded_files(tuple(file_paths))
        if vectorstore:
            st.session_state.chain = build_conversational_chain(vectorstore)
            st.session_state.indexed_files = file_key
            st.sidebar.success(f'{len(uploaded)} file(s) indexed!')
```

PYTHON — Part F (Main UI, Part 2)

```
# Display chat history
for msg in st.session_state.messages:
    role_class = 'chat-user' if msg['role'] == 'user' else 'chat-bot'
    icon = 'User' if msg['role'] == 'user' else 'Bot'
    st.markdown(
        f'<div class="{role_class}">{icon}: {msg["content"]}</div>',
        unsafe_allow_html=True
    )

# Query input
if query := st.chat_input('Ask a question about your documents...'):
    if not st.session_state.chain:
        st.warning('Please upload documents first!')
    else:
        st.session_state.messages.append({'role': 'user', 'content': query})

        with st.spinner('Retrieving and answering...'):
            result = st.session_state.chain.invoke({'question': query})
            answer = result['answer']
            sources = result.get('source_documents', [])

        st.session_state.messages.append({'role': 'assistant', 'content': answer})

        if show_sources and sources:
            with st.expander(f'Source Chunks ({len(sources)} retrieved)':
                for i, doc in enumerate(sources):
                    st.markdown('<div class="source-box">', unsafe_allow_html=True)
                    st.write(f'Chunk {i+1} | Source: {doc.metadata.get("source","?")}',
                            f'| Page: {doc.metadata.get("page","?")}')
                    st.write(doc.page_content)
                    st.markdown('</div>', unsafe_allow_html=True)

st.rerun()
```

4.4 Run the Streamlit App

BASH

```
cd ~/rag-labs/lab3_streamlit_qa

# Run Streamlit – binds to all interfaces so EC2 can be accessed remotely
streamlit run app.py --server.port 8501 --server.address 0.0.0.0
```

■ **NOTE**

Access the app in your browser at: <http://<YOUR-EC2-PUBLIC-IP>:8501>

Make sure port 8501 is open in your EC2 Security Group (Inbound Rules):

Type: Custom TCP | Port: 8501 | Source: 0.0.0.0/0

MODULE 04

LAB 4 — Multi-Model Answer Comparison

Compare GPT-4o, Claude 3.5, and Gemini 1.5 on identical RAG queries

5.1 Why Compare Models?

Different LLMs have different strengths: GPT-4o tends to be precise and instruction-following; Claude tends to be verbose and careful about uncertainty; Gemini tends to be concise with strong reasoning. By running the **same RAG pipeline** with different LLM backends, participants can observe how model choice affects answer quality, tone, and completeness.

Dimension	OpenAI GPT-4o-mini	Anthropic Claude 3 Haiku	Google Gemini 1.5 Flash
Answer length	Concise, precise	Detailed, nuanced	Moderate, structured
Uncertainty handling	States clearly	Extensive caveats	Direct
Instruction following	Excellent	Very good	Good
Cost (approx)	\$0.15/1M input tokens	\$0.25/1M input tokens	\$0.075/1M input tokens
Speed	Fast	Fast	Very fast

5.2 Setup

BASH

```
mkdir -p ~/rag-labs/lab4_model_comparison
cd ~/rag-labs/lab4_model_comparison
cp -r ../lab1_rag_pipeline/documents .
cp ../lab1_rag_pipeline/config.py .
```

5.3 model_clients.py — Unified API Wrappers

PYTHON — model_clients.py

```
# model_clients.py
import os
from dotenv import load_dotenv
load_dotenv(dotenv_path='../.env')

from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

def get_openai_llm():
    return ChatOpenAI(
        model='gpt-4o-mini',
        temperature=0,
        api_key=os.getenv('OPENAI_API_KEY')
    )

def get_claude_llm():
    return ChatAnthropic(
        model='claude-3-haiku-20240307',
        temperature=0,
        api_key=os.getenv('ANTHROPIC_API_KEY')
    )

def get_gemini_llm():
    return ChatGoogleGenerativeAI(
        model='gemini-1.5-flash',
        temperature=0,
        google_api_key=os.getenv('GOOGLE_API_KEY')
    )

MODELS = {
    'GPT-4o-mini (OpenAI)': get_openai_llm,
    'Claude 3 Haiku (Anthropic)': get_claude_llm,
    'Gemini 1.5 Flash (Google)': get_gemini_llm,
}
```

Install Anthropic and Google LangChain adapters

BASH

```
pip install langchain-anthropic langchain-google-genai
```

5.4 compare_models.py — CLI Comparison Script

PYTHON — compare_models.py Part 1

```
# compare_models.py
import time
from config import *
from model_clients import MODELS

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

PROMPT = PromptTemplate(
    template='''Answer based only on the context below.
    If you cannot answer from the context, say so clearly.

    Context: {context}
    Question: {question}
    Answer: ''',
    input_variables=['context', 'question']
)

def build_vectorstore():
    loader = DirectoryLoader('documents', glob='**/*.txt',
                             loader_cls=TextLoader,
                             loader_kwargs={'encoding': 'utf-8'})

    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP
    )
    chunks = splitter.split_documents(docs)
    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL,
                                   openai_api_key=OPENAI_API_KEY)
    return FAISS.from_documents(chunks, embeddings)
```

PYTHON — compare_models.py Part 2

```
def compare_all_models(query: str, vectorstore):
    print(f'QUERY: {query}')
    results = {}
    for model_name, get_llm_fn in MODELS.items():
        print(f'--- {model_name} ---')
        try:
            llm = get_llm_fn()
            chain = RetrievalQA.from_chain_type(
                llm=llm, chain_type='stuff',
                retriever=vectorstore.as_retriever(
                    search_kwargs={'k': TOP_K}),
                return_source_documents=False,
                chain_type_kwargs={'prompt': PROMPT}
            )
            start = time.time()
            result = chain.invoke({'query': query})
            elapsed = time.time() - start
            answer = result['result']
            results[model_name] = {'answer': answer, 'time': elapsed}
            print(f'Answer ({elapsed:.2f}s): {answer}')
        except Exception as e:
            print(f'ERROR: {e}')
            results[model_name] = {'answer': f'ERROR: {e}', 'time': 0}
    return results

if __name__ == '__main__':
    vectorstore = build_vectorstore()
    queries = [
        'How many sick leave days are employees entitled to?',
        'What password requirements are specified?',
        'Who should I contact about a security incident?',
        'What is the meal allowance per day?',
        'Is annual leave policy mentioned in the documents?',
    ]
    for q in queries:
        compare_all_models(q, vectorstore)
```

BASH

```
python compare_models.py
```

5.5 comparison_app.py — Streamlit Side-by-Side UI

PYTHON — comparison_app.py Part 1

```
# comparison_app.py
import streamlit as st
import time
from config import *
from model_clients import MODELS
from compare_models import build_vectorstore
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

st.set_page_config(page_title='Model Comparison', layout='wide')
st.title('Multi-Model RAG Comparison')
st.markdown('Compare **GPT-4o**, **Claude**, and **Gemini** on the same query.')

PROMPT = PromptTemplate(
    template='Context: {context}\n\nQuestion: {question}\n\nAnswer:',
    input_variables=['context', 'question']
)

@st.cache_resource(show_spinner='Building vector index...')
def get_vectorstore():
    return build_vectorstore()

vectorstore = get_vectorstore()

query = st.text_input(
    'Enter your question:',
    placeholder='How many sick leave days are employees entitled to?'
)
```

PYTHON — comparison_app.py Part 2

```
if st.button('Compare All Models', type='primary') and query:
    cols = st.columns(len(MODELS))

    for col, (model_name, get_llm_fn) in zip(cols, MODELS.items()):
        with col:
            st.markdown(f'### {model_name}')
            with st.spinner('Querying...'):
                try:
                    llm = get_llm_fn()
                    chain = RetrievalQA.from_chain_type(
                        llm=llm, chain_type='stuff',
                        retriever=vectorstore.as_retriever(
                            search_kwargs={'k': TOP_K}),
                        return_source_documents=False,
                        chain_type_kwargs={'prompt': PROMPT}
                    )
                    t0 = time.time()
                    result = chain.invoke({'query': query})
                    elapsed = time.time() - t0
                    st.success(result['result'])
                    st.caption(f'Time: {elapsed:.2f}s')
                except Exception as e:
                    st.error(f'Error: {e}')
```

BASH

```
streamlit run comparison_app.py --server.port 8502 --server.address 0.0.0.0
```

■ NOTE

Access comparison UI at: <http://<YOUR-EC2-PUBLIC-IP>:8502>

Open port 8502 in your EC2 Security Group (same way as port 8501 in Lab 3).

Troubleshooting Guide

Common errors across all labs and their solutions

Error / Symptom	Likely Cause	Solution
AuthenticationError: Incorrect API key	Wrong or missing OPENAI_API_KEY	Check .env file; run: echo \$OPENAI_API_KEY to verify it's loaded
ModuleNotFoundError: No module named 'langchain'	Virtual environment not activated or packages not installed	Run: source ~/rag-labs/venv/bin/activate && pip install langchain
ValueError: allow_dangerous_deserialization	FAISS load requires explicit trust flag in newer LangChain	Add allow_dangerous_deserialization=True to FAISS.load_local()
ConnectionRefusedError on Streamlit	Port not open in EC2 Security Group	Add inbound rule: TCP port 8501/8502, Source: 0.0.0.0/0
Empty retrieval / no source documents	Score threshold too high or documents not indexed	Lower score_threshold or rebuild index with force rebuild
langchain_anthropic not found	Package not installed	pip install langchain-anthropic
RateLimitError from OpenAI	Too many API calls in a short window	Add time.sleep(1) between calls or use a higher-tier API plan
FAISS IndexFlatL2: dimension mismatch	Embedding model changed between index build and load	Delete faiss_index/ directory and rebuild from scratch

Quick Diagnostic Commands

BASH — Diagnostics

```
# Check all packages are installed
pip list | grep -E 'langchain|faiss|openai|streamlit|anthropic|google'

# Test OpenAI API connectivity
python -c "
from openai import OpenAI; import os; from dotenv import load_dotenv
load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))
r = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{'role':'user', 'content':'Say OK'}],
    max_tokens=5
)
print('OpenAI OK:', r.choices[0].message.content)
"

# Verify FAISS index was created
ls -lh ~/rag-labs/lab1_rag_pipeline/faiss_index/

# Check EC2 open ports
sudo ss -tlnp | grep -E '8501|8502'
```

Capstone Challenge

Integrate all four labs into a production-grade RAG system

Objective

Build a **unified Document Intelligence Platform** that combines: full RAG pipeline (Lab 1), retrieval quality analysis (Lab 2), Streamlit UI (Lab 3), and multi-model comparison (Lab 4) — all in a single app.

Challenge	Requirement	Points
Bronze	Upload a real PDF → build index → answer 5 questions accurately	20
Silver	Add metadata filtering by document type (policy / handbook / other)	20
Gold	Show side-by-side comparison of 3 models for every query in the UI	30
Platinum	Add retrieval quality score gauge and show MMR vs similarity results	30

✓ TIP

Submission: Share your EC2 IP with the trainer for live demo review.

Bonus: Deploy the Streamlit app with HTTPS using ngrok (pip install pyngrok) for a public URL.